

Virtual Cinematography with Unreal Engine

Learn how Unreal Engine's virtual cinematography framework has given rise to new and advanced techniques in film and animation studios.

Thomas Kilkenny
Epic Games, USA
thomas.kilkenny@epicgames.com

Adam Kunsberg
Independent, USA
adamkunsberg@gmail.com

John Brennan
Walt Disney Studios, USA
john.brennan@disneympp.com

ABSTRACT

This talk focuses on the development and real-world applications of Virtual Cinematography tools, techniques, and workflows using Unreal Engine and accompanying real time technologies. These techniques enable the creation of highly customizable, handheld, point-and-shoot Virtual Cameras (VCams), which can be operated similarly to a live-action camera. Though in a state of constant refinement over the course of nearly two decades, the primary goal has remained consistent: recreate the feel of a cinema camera and make applicable the hard-earned skills of live-action filmmakers and creatives when working in the hybrid digital and animated space of virtual production.

VCam has traditionally been a highly specific process created for a particular filmmaker by a highly skilled technical team on a given production. This work seeks to make these workflows available to all filmmakers at any level and to share the developments occurring in Unreal Engine and active productions today.

KEYWORDS

Virtual Camera, VCam, Virtual Production, Virtual Cinematography, Real-time Filmmaking, Simulated Worlds

ACM Reference Format:

Thomas Kilkenny, Adam Kunsberg, and John Brennan. 2023. Virtual Cinematography with Unreal Engine: Lean how Unreal Engine's virtual cinematography framework has given rise to new and advanced techniques in film and animation studios.. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH '23 Talks)*, August 06–10, 2023, Los Angeles, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3587421.3604476>

1 INTRODUCTION

The virtual camera - and virtual cinematography itself - has traditionally involved specialized, expensive hardware and sequestered workflows on flagship virtual production films. With the popularization of this discipline in a readily accessible form, through the combination of engine software and consumer hardware, Unreal Engine's new framework for VCams has encouraged professional studios to innovate on their previous virtual cinematography workflows.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '23 Talks, August 06–10, 2023, Los Angeles, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0143-6/23/08.

<https://doi.org/10.1145/3587421.3604476>

Traditionally, the core elements of virtual cameras consist of tracking, device inputs, and a screen. The device inputs would be layered into the tracking data to produce a final state for the camera, which would be sent to a real-time renderer and displayed on a screen. In previous workflows, these calculations were done by an external application which then passed the final state of the virtual camera to the real-time render. With Unreal Engine's VCam framework, we can pass all and any inputs directly to the game engine, and evaluate them within the context of the scene the virtual camera is placed in. This allows for more advanced techniques that leverage a game engine's ability to simulate physics, collisions, and dynamic animations, all while reconciling the user's inputs in real-time.

2 VCAM CORE

Unreal Engine's VCam framework (VCam Core) is built as an open canvas where any part of the scene or engine can contribute to the state of the VCam. This includes user inputs, device inputs, tracking, animations, physical simulations, editor assets, and anything the engine has access to. In addition, VCam Core provides a series of tools for defining the functionality, behavior, appearance, and I/O of the virtual camera within this context.

- *Live Link*. Physical position, calculated through the tracking system of your choice, is transmitted directly to the engine through the Live Link protocol. This position is used not only for the position to be applied to the VCam but can also be intercepted by other engine systems and custom tools, allowing for increasingly complex dynamic responses.
- *Enhanced Input*. Non-tracking input is routed to the engine via the Enhanced Input system. Any devices recognized by the OS are accessible as input events in Enhanced Input and can be used to provide human access to the camera and supporting tool sets through a variety of hardware devices.
- *The Modifier Stack*. Each camera and its available tools is constructed of a series of modular, visual scripting based tools named modifiers, which accept input evaluation in a stacked order. Modifiers have full access to the various inputs into the VCam core as well as to the full API of the engine. These can range from the application of smoothing to Live Link data to toolsets for interacting with the sequence timeline to management of constraints.
- *Output Providers*. Output providers define the output rules for each VCam and the accompanying HUD for each output. These outputs can range from SDI to Pixel Streaming to custom protocols. One camera supports multiple simultaneous outputs, with a separate HUD drawn to each.

- *Unreal Motion Graphics HUDs*. The UI for the Virtual Camera is user defined through Unreal Motion Graphics (UMG) and streamed along with the plate output.
- *Multi-User*. Multi-User (MU) acts as a collaboration toolset for multiple instances of Unreal Engine, allowing for simultaneous manipulation of the virtual scene by separate operators in real-time. Utilization of Multi-User in a virtual camera scene allows for not only the camera to respond to not only the human inputs and engine systems running on the same instance but also to those running on other instances in the MU session. In the context of a VCam shoot, MU allows for real-time multi-cam on a single simulated scene as well as for the simultaneous collaboration of large film crews including dolly grips, lighting crew, camera assistants, set decoration, and other members of the shooting crew, each with their own machine, role, and view of the scene.

3 ADVANCED TECHNIQUES

Utilizing Unreal Engine's VCam Core framework, film and animation studios have pioneered advanced toolsets that leverage the combination of traditional virtual camera input systems with the full range of Unreal Engine's real-time modules.

1. *Dynamic Constraints*. Operators can manipulate the target and strength of constraints in real-time as they are recording virtual cameras. This allows them the opportunity to dynamically respond to the subjects of each performance as they are shooting.
2. *Collision Traces and Physics*. The virtual camera's final position can be modified by the objects in the scene such as terrain, walls, and characters. More complex uses of collision traces allows for interactive systems that mimic real-life machinery like mounting cameras on moving vehicles, cable-cam systems, and other equipment used on live-action sets that contribute to the visual language.
3. *Dynamic Animation*. Unreal Engine's Sequencer, Control Rig, and Animation Blueprinting system are used to build an operator friendly toolset that allows them to solve per-shot performance cheats and problems such as stacked characters, eye-lines, scene extensions, and beat modifications, all in real-time without slowing down the shoot.
4. *Interactive Motion*. By visualizing Unreal Engine's sequence data within the virtual environment, we empower the camera operator to interact and modify previous takes without the need to use a traditional curve editor. This intuitive process allows live-action camera operators to take part in a workflow that was previously only available within an animation context.

4 CONCLUSION

With the increased adoption of game engines within virtual productions, virtual camera workflows have ventured deeper in their interactions with non-camera systems. In doing so, virtual camera workflows have grown to embrace more advanced techniques that leverage the features of game engines beyond their real-time rendering capabilities. Collisions, physics simulations, multi-user, and other game engine interaction systems can be used to increase

speed, accuracy, and creative freedom when creating films and animations. This work represents only the early stages of the ongoing efforts to better emulate the live action process. It is our hope that with continued development and cooperation within the Virtual Production and VFX communities, the transition of existing and yet discovered creators into the virtual world can be made more intuitive and effortless.